

Processamento de Alto Desempenho (PAD)

Relatório de Trabalho 1 – Distância entre Vetores
Implementação, otimização e análise experimental de desempenho

Xavier Paulino Sebastião

Agosto de 2026

Resumo

Este trabalho avalia o cálculo da distância quadrática entre um vetor de referência e um conjunto de vetores reais e compara a implementação base com uma versão otimizada do mesmo kernel. A versão base utiliza um laço escalar. A versão otimizada emprega SIMD AVX, desenrolamento do laço, quatro acumuladores vetoriais independentes e qualificadores `restrict`, mantendo a execução em um único fluxo e as mesmas principais opções de compilação. Foram avaliados oito tamanhos de vetor, de $T = 32$ a $T = 4096$, com $N = 16384$ mantido constante na comparação. A versão otimizada apresentou menor tempo mediano em todos os tamanhos avaliados. O fator de aceleração variou aproximadamente entre 1,22 e 1,73, com maior ganho em $T = 1024$. Em $T = 4096$, o fator permaneceu acima de 1, em aproximadamente 1,50. Os resultados mostram que a reorganização do kernel aumenta a vazão e reduz o custo por elemento, embora a magnitude do ganho dependa do tamanho da entrada.

1 Introdução

O problema consiste em calcular, para cada vetor x_i de um conjunto X com N vetores, a distância quadrática em relação a um vetor de referência q . Cada vetor contém T valores reais, e a distância é definida por

$$D(q, x_i) = \sum_{j=0}^{T-1} (q_j - x_{ij})^2, \quad i = 0, \dots, N - 1. \quad (1)$$

Cada resultado é armazenado após o processamento do respectivo vetor. Mantendo N constante e variando T , a quantidade de elementos processados por execução é $N \times T$, de modo que o trabalho aritmético cresce linearmente com T .

A primeira etapa do trabalho caracterizou o comportamento da implementação escalar para diferentes valores de T . A segunda etapa introduziu uma versão otimizada do mesmo cálculo e comparou diretamente tempo de execução, taxas de processamento, custo por elemento, fator de aceleração e variabilidade. A comparação foi realizada com múltiplas aferições, pois medições isoladas podem ser afetadas por variações transitórias da plataforma [2, 3, 8].

2 Implementações avaliadas

2.1 Versão base

A implementação de referência foi desenvolvida em C++ [7], com valores do tipo *double* e armazenamento contíguo da matriz X . Para cada vetor, o kernel percorre sequencialmente os T elementos e acumula a soma dos quadrados das diferenças. Conceitualmente, o núcleo computacional avaliado corresponde ao seguinte trecho:

```

for (size_t i = 0; i < N; ++i) {
    const double* row = X + i * T;
    double sum = 0.0;
    for (size_t j = 0; j < T; ++j) {
        const double d = q[j] - row[j];
        sum += d * d;
    }
    distances[i] = sum;
}

```

O kernel base foi mantido no mesmo programa para permitir medir as duas versões sob o mesmo procedimento experimental e evitar depender apenas de resultados obtidos em momentos diferentes.

2.2 Versão otimizada

A versão otimizada preserva a mesma estrutura de dados e realiza o mesmo cálculo matemático da versão base. A execução permanece restrita a um único fluxo, portanto, o ganho de desempenho não decorre do uso de múltiplos núcleos. A otimização concentra-se no laço interno responsável pelo cálculo da distância. Os ponteiros são declarados com `__restrict__`, fornecendo ao compilador a informação de que as regiões de memória referenciadas não se sobrepõem. Quando o suporte a AVX está disponível, quatro valores do tipo *double* são processados por instrução vetorial. O laço é desenrolado em blocos de 16 elementos e utiliza quatro acumuladores AVX independentes. Essas modificações permitiram alterar a forma de execução do cálculo, mas não o cálculo realizado ou seu resultado.

O trecho central pode ser representado por:

```

__m256d acc0 = _mm256_setzero_pd();
__m256d acc1 = _mm256_setzero_pd();
__m256d acc2 = _mm256_setzero_pd();
__m256d acc3 = _mm256_setzero_pd();

for (; j + 15 < T; j += 16) {
    // quatro blocos de 4 doubles
    // carga, subtracao, multiplicacao e acumulacao
}

```

Os elementos remanescentes são tratados por um bloco AVX de quatro elementos e, quando necessário, por um trecho escalar final. Assim, a implementação não exige que T seja múltiplo de 16. Como a ordem das somas difere da versão escalar, a verificação utiliza tolerância numérica, em vez de igualdade bit a bit. A compilação mantém `C++20`, `-O3` e `-march=native`, sem `-ffast-math`. O uso de SIMD e as transformações de laço estão diretamente relacionados à exploração da microarquitetura do processador [4].

2.3 Interface de execução

O tamanho do vetor T é informado como primeiro argumento posicional do executável. Caso sejam utilizados parâmetros adicionais, eles são informados após T . Na forma de execução destinada à avaliação, o programa utiliza a versão otimizada, mantém $N = 16384$ e realiza cinco execuções de aquecimento não cronometradas antes da única medição reportada. A saída padrão contém exclusivamente uma linha no formato

```
xavier, T, tempo_ms
```

em que o primeiro campo identifica o programa, o segundo informa o tamanho do vetor e o terceiro corresponde ao tempo de execução em milissegundos. Alocação, inicialização dos dados,

aquecimento e impressão permanecem fora da região cronometrada. Essa interface de execução é distinta do procedimento comparativo descrito na Seção 3.3, no qual os scripts experimentais utilizam a configuração congelada e duas execuções de aquecimento por combinação em cada bloco.

3 Metodologia experimental

3.1 Plataforma e ambiente

As aferições foram executadas em um Lenovo ThinkPad T530 equipado com processador Intel Core i5-3320M a 2,60 GHz, arquitetura x86-64, família 6, modelo 58. A máquina possui dois núcleos físicos e quatro CPUs lógicas, com dois fluxos por núcleo, e disponibiliza AVX, SSE4.1 e SSE4.2. A compilação foi realizada com GCC 10.5.0, usando `/usr/bin/g++` e as opções `-O3 -march=native`. O sistema operacional utilizado foi Ubuntu 24.04.4 LTS, com kernel Linux 7.0.0-30-generic. O processo experimental foi fixado no CPU lógico 2, selecionado deterministicamente como representante do segundo núcleo físico entre os CPUs permitidos; seu sibling SMT é o CPU lógico 3. Durante o registro do ambiente, o governador de frequência era `schedutil` e o turbo permanecia habilitado. Os scripts de análise e geração dos gráficos foram executados com Python 3.12.

3.2 Calibração da configuração experimental

Antes da comparação entre as implementações, N e o CPU utilizado foram definidos por uma etapa de calibração executada separadamente das medições comparativas. O procedimento selecionou $N = 16384$, mantendo o critério de escolher uma potência de dois compatível com o limite de memória estabelecido para o maior tamanho avaliado. Para $T = 4096$, a principal alocação estimada no pior caso foi de aproximadamente 512,2 MiB.

A seleção do CPU foi realizada de forma determinística. Entre os CPUs lógicos permitidos $\{0, 1, 2, 3\}$, os representantes de núcleos físicos foram 0 e 2; o CPU lógico 2 foi selecionado e possui o CPU lógico 3 como sibling SMT.

A calibração também avaliou a adequação da temporização com `CLOCK_MONOTONIC_RAW` [1]. A resolução nominal reportada foi de 1 ns. Para $T = 32$, a mediana do tempo do kernel foi de 0,441797 ms, enquanto a mediana entre duas leituras consecutivas do relógio foi de 28 ns. A razão entre esse intervalo e o tempo mediano do kernel foi de aproximadamente 0,006338%, abaixo do limite de 1% adotado pelo procedimento. A calibração foi, portanto, aceita e a configuração com $N = 16384$, *seed* 42 e CPU lógico 2 foi congelada para os experimentos.

3.3 Procedimento comparativo

As versões base e otimizada foram avaliadas no mesmo procedimento experimental. Em cada bloco, as 16 combinações formadas por duas implementações e oito valores de T foram ordenadas de forma pseudoaleatória e determinística, com semente base 42. Esse desenho foi adotado para reduzir a associação sistemática entre uma implementação e um momento específico da execução, uma vez que detalhes da ordem experimental podem influenciar resultados de desempenho [6].

Nos resultados apresentados, $N = 16384$ permaneceu constante para as duas versões e para todos os valores

$$T \in \{32, 64, 128, 256, 512, 1024, 2048, 4096\}.$$

Cada combinação (implementação, T) foi aferida 30 vezes, distribuídas em cinco blocos de seis repetições. Em cada bloco, cada combinação foi precedida por duas execuções de aquecimento não cronometradas. A mediana foi utilizada como medida central, enquanto as 30 observações foram preservadas para caracterizar a variabilidade [8].

A região cronometrada contém exclusivamente o cálculo e o armazenamento das N distâncias. Alocação, geração e inicialização dos dados, execuções de aquecimento, cálculo do checksum, escrita de arquivos e apresentação dos resultados permanecem fora do intervalo medido. A temporização utiliza `CLOCK_MONOTONIC_RAW`. A compilação emprega `/usr/bin/g++` com `-O3 -march=native`.

3.4 Índices de desempenho

Além do tempo de execução t , foram calculadas as taxas

$$P_v = \frac{N}{t}, \quad P_e = \frac{NT}{t}, \quad (2)$$

onde P_v representa vetores processados por segundo e P_e representa elementos processados por segundo. O custo temporal por elemento é dado por

$$C_e = \frac{t \times 10^9}{NT} \text{ ns/elemento}. \quad (3)$$

O fator de aceleração da versão otimizada em relação à versão base é

$$S(T) = \frac{t_{\text{base}}(T)}{t_{\text{otimizada}}(T)}. \quad (4)$$

Valores de $S(T)$ superiores a 1 indicam redução do tempo de execução. A redução percentual é expressa por

$$R(T) = 100 \left(1 - \frac{t_{\text{otimizada}}(T)}{t_{\text{base}}(T)} \right). \quad (5)$$

4 Resultados

4.1 Resumo quantitativo

A Tabela 1 reúne as principais métricas obtidas na execução experimental. A versão otimizada apresentou menor tempo mediano e menor custo por elemento em todos os tamanhos avaliados. O menor fator de aceleração foi observado em $T = 32$, com aproximadamente 1,22, e o maior em $T = 1024$, com cerca de 1,73. Em $T = 4096$, o fator ficou em aproximadamente 1,50.

Tabela 1: Comparação entre a versão base e a versão otimizada para $N = 16384$.

T	Tempo mediano (ms)		P_e (10^9 elem./s)		ns/elem.		Aceleração	Redução (%)
	Base	Otimizada	Base	Otimizada	Base	Otimizada		
32	0,478	0,392	1,098	1,336	0,911	0,749	1,22	17,8
64	1,047	0,805	1,001	1,302	0,999	0,768	1,30	23,1
128	2,160	1,503	0,971	1,395	1,030	0,717	1,44	30,4
256	4,708	2,907	0,891	1,443	1,123	0,693	1,62	38,3
512	9,373	5,817	0,895	1,442	1,117	0,693	1,61	37,9
1024	19,227	11,092	0,873	1,513	1,146	0,661	1,73	42,3
2048	40,187	27,750	0,835	1,209	1,198	0,827	1,45	30,9
4096	77,485	51,536	0,866	1,302	1,155	0,768	1,50	33,5

4.2 Tempo de execução

A Figura 1 mostra que o tempo mediano aumenta com T nas duas implementações, enquanto a curva da versão otimizada permanece abaixo da versão base em toda a faixa. Em $T = 32$, os tempos medianos foram 0,478 ms na versão base e 0,392 ms na otimizada. Em $T = 4096$, passaram

para 77,485 ms e 51,536 ms, respectivamente. A diferença entre as versões torna-se maior nos tamanhos mais elevados, embora o ganho relativo não permaneça constante.

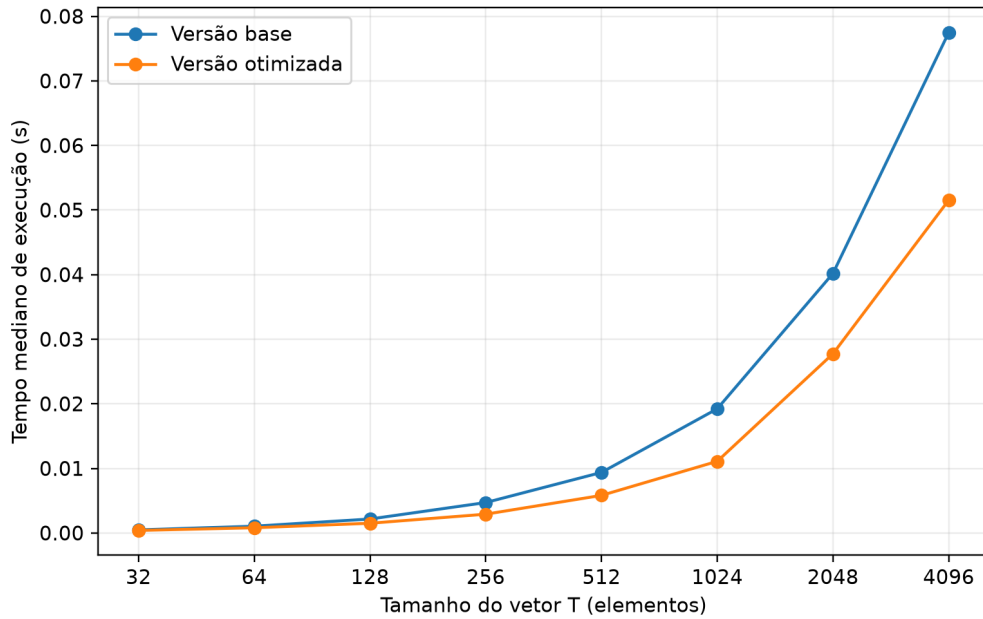


Figura 1: Tempo mediano de execução das versões base e otimizada.

4.3 Taxas de processamento

A taxa de vetores processados por segundo é maior na versão otimizada em todos os tamanhos, como mostra a Figura 2. Em $T = 32$, as medianas foram aproximadamente 34,31 milhões de vetores/s na versão base e 41,75 milhões na otimizada. Em $T = 4096$, reduziram-se para aproximadamente 0,211 e 0,318 milhão de vetores/s, respectivamente. Essa queda é esperada porque cada vetor exige mais trabalho à medida que T aumenta.

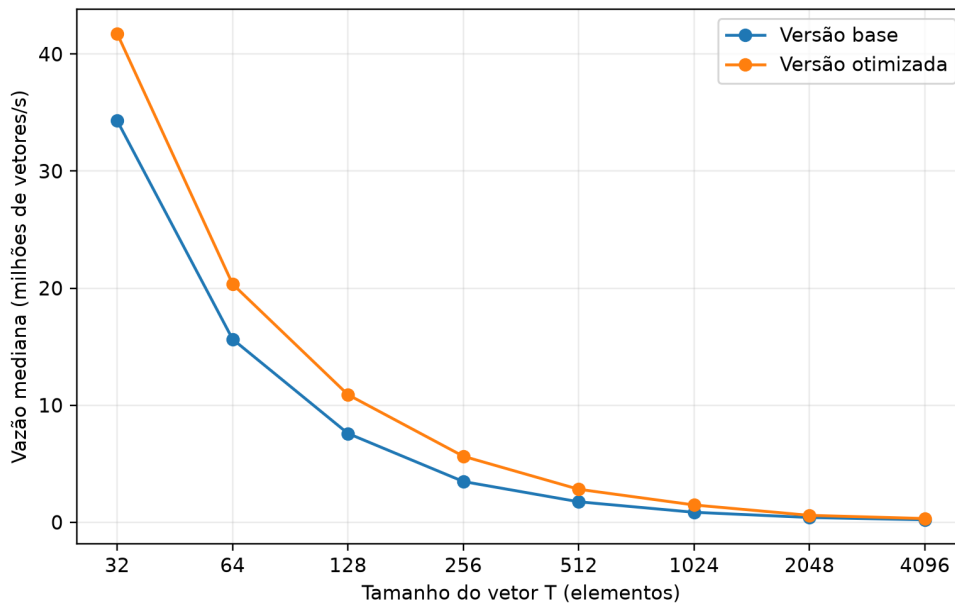


Figura 2: Vazão mediana de vetores das duas versões.

A Figura 3 apresenta a vazão em elementos por segundo. Na versão base, a mediana diminui de aproximadamente 1,098 bilhão de elementos/s em $T = 32$ para 0,866 bilhão em $T = 4096$,

com o menor valor, cerca de 0,835 bilhão, em $T = 2048$. Na versão otimizada, a vazão cresce de aproximadamente 1,336 bilhão em $T = 32$ até o máximo de 1,513 bilhão em $T = 1024$. Em $T = 2048$, entretanto, cai para aproximadamente 1,209 bilhão e recupera-se parcialmente para 1,302 bilhão em $T = 4096$.

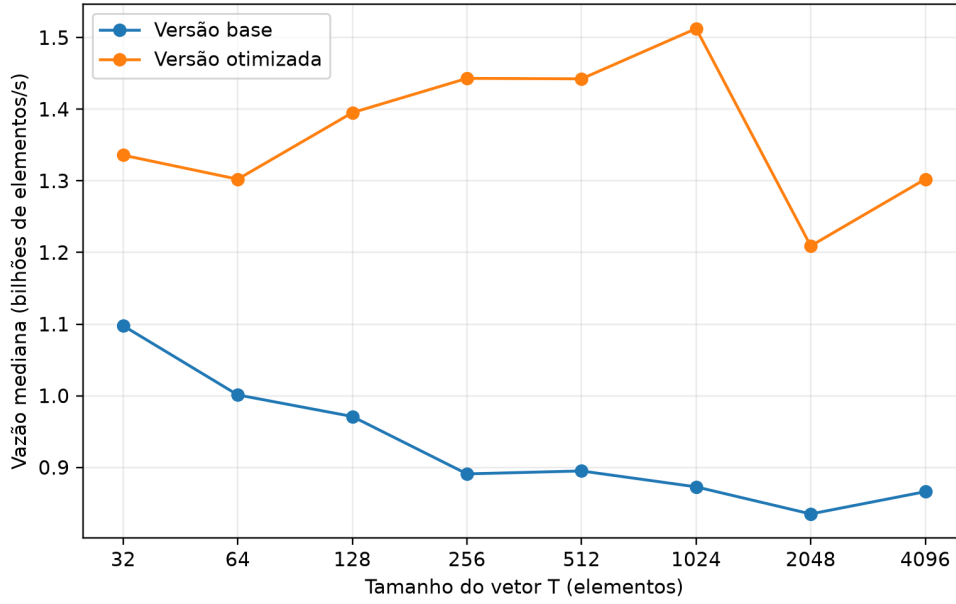


Figura 3: Vazão mediana de elementos das duas versões.

4.4 Custo por elemento

O custo por elemento fornece a informação inversa de P_e . Na versão base, varia de aproximadamente 0,911 ns/elemento em $T = 32$ a 1,198 ns/elemento em $T = 2048$, recuando para 1,155 ns/elemento em $T = 4096$. Na versão otimizada, o custo diminui até o mínimo de 0,661 ns/elemento em $T = 1024$, aumenta para 0,827 ns/elemento em $T = 2048$ e volta a 0,768 ns/elemento em $T = 4096$.

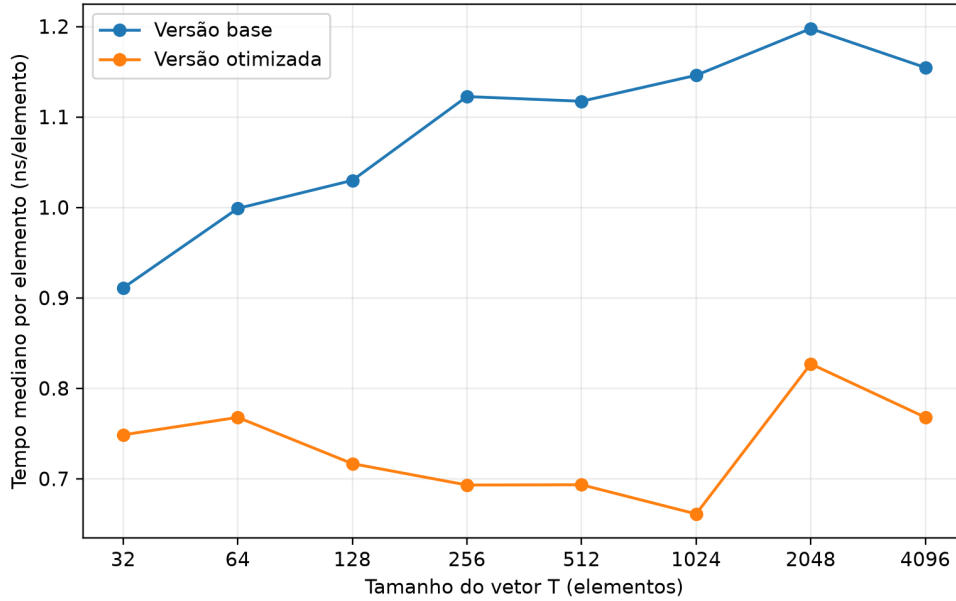


Figura 4: Tempo mediano por elemento nas duas versões.

A versão otimizada permanece mais eficiente por elemento em toda a faixa. A mudança entre

$T = 1024$ e $T = 2048$ é particularmente mais visível onde a vazão otimizada diminui de 1,513 para 1,209 bilhão de elementos/s e o custo por elemento aumenta de 0,661 para 0,827 ns/elemento. Os tempos medidos, isoladamente, não permitem determinar se essa mudança decorre de cache, memória principal, prefetch, frequência do processador ou outro efeito microarquitetural. Nesse caso, a infraestrutura `perf_event` do Linux poderia oferecer eventos como ciclos, instruções e falhas de cache, mas esses indicadores não integram os dados avaliados [5].

4.5 Fator de aceleração

A Figura 5 resume o ganho relativo. Todos os valores permanecem acima de 1, demonstrando que a versão otimizada foi mais rápida em cada tamanho avaliado. O fator cresce de aproximadamente 1,22 em $T = 32$ para 1,62 em $T = 256$, permanece próximo desse valor em $T = 512$ e atinge o máximo de aproximadamente 1,73 em $T = 1024$. Em $T = 2048$, diminui para cerca de 1,45 e recupera-se parcialmente para aproximadamente 1,50 em $T = 4096$.

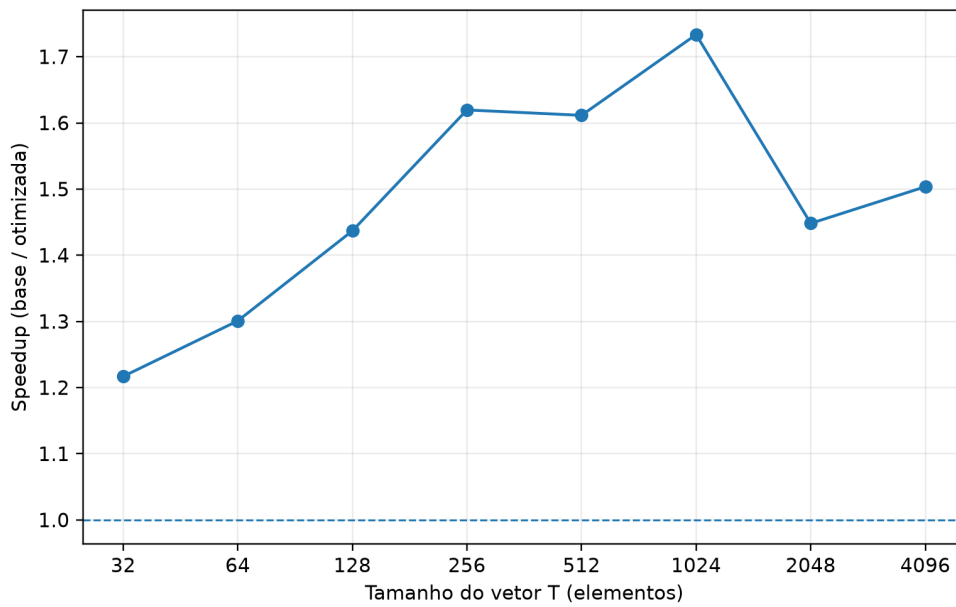


Figura 5: Fator de aceleração da versão otimizada em relação à versão base. A linha tracejada representa ausência de ganho.

4.6 Variabilidade

A Figura 6 apresenta as 30 aferições de cada combinação. A dispersão absoluta torna-se mais visível nos maiores tamanhos, embora a variabilidade relativa não seja monotônica. Em $T = 4096$, a versão base apresentou tempos entre aproximadamente 70,054 e 91,312 ms, enquanto a otimizada variou entre 43,522 e 87,859 ms. Em $T = 2048$, a versão otimizada também apresentou amplitude elevada, de aproximadamente 19,671 a 52,388 ms. As observações discrepantes foram preservadas porque não há evidência independente de erro que justifique sua remoção. Esse comportamento reforça a necessidade de considerar a distribuição das medições, e não apenas uma única execução.

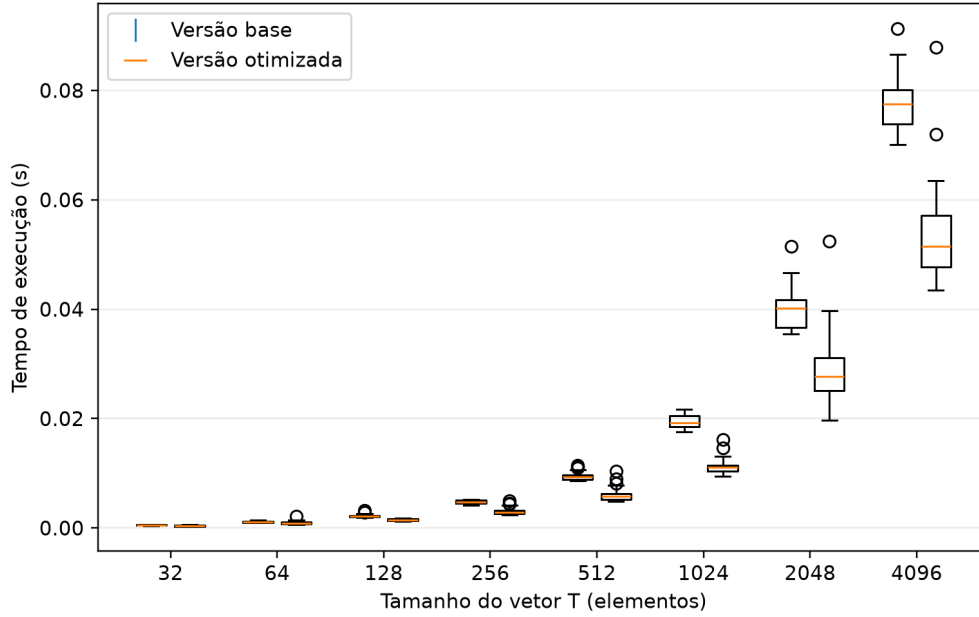


Figura 6: Distribuição dos tempos de execução das versões base e otimizada.

5 Discussão

Os resultados mostram que a otimização melhora o desempenho em toda a faixa avaliada, mas o ganho varia com T . A versão base utiliza uma única variável de acumulação, criando dependência entre iterações sucessivas. A versão otimizada distribui o trabalho entre quatro acumuladores AVX independentes e processa vários valores simultaneamente. Essa organização aumenta o paralelismo em nível de instrução e reduz a dependência sequencial da soma. O desenrolamento do laço também pode ter influenciado a diminuição da frequência relativa das operações de controle.

O fator de aceleração permanece abaixo de 2 em todos os tamanhos, apesar de um registrador AVX de 256 bits comportar quatro valores `double`. Isto acontece porque a capacidade de processar quatro elementos por instrução vetorial não se pode traduzir diretamente em um ganho de 4 vezes, pois o tempo global também inclui carregamentos, redução dos acumuladores, controle do laço, latências das instruções e movimentação de dados. Além disso, a versão base foi compilada com `-O3 -march=native`, que pode permitir ao compilador aplicar algumas otimizações disponíveis para o processador.

O maior fator de aceleração ocorre em $T = 1024$, com aproximadamente 1,73, correspondendo a redução de 42,3% no tempo mediano. Nesse mesmo tamanho, a versão otimizada alcança sua maior vazão de elementos, aproximadamente 1,513 bilhão de elementos/s, e seu menor custo por elemento, 0,661 ns/elemento. Em $T = 2048$, há uma mudança marcada: o fator de aceleração cai para aproximadamente 1,45, a vazão otimizada diminui para 1,209 bilhão de elementos/s e o custo aumenta para 0,827 ns/elemento. Em $T = 4096$, essas métricas apresentam recuperação parcial, com fator de aceleração de aproximadamente 1,50.

A ocorrência simultânea dessas mudanças mostra que o ganho não depende apenas da quantidade nominal de operações. A hierarquia de memória e o prefetch de hardware podem influenciar percursos sequenciais desse tipo [4]; entretanto, os dados disponíveis não incluem contadores de cache, largura de banda de memória, ciclos, instruções ou frequência instantânea. Portanto, cache, memória, prefetch e frequência permanecem hipóteses plausíveis, mas não podem ser identificados como causas a partir dos tempos coletados.

A comparação foi estruturada para reduzir efeitos temporais da máquina. As duas implementações foram medidas no mesmo procedimento, em cinco blocos, com 30 aferições por combinação e ordem pseudoaleatória determinística, em vez de comparar apenas conjuntos independentes

obtidos em momentos distintos. Esse cuidado segue recomendações de benchmarking que enfatizam repetição, controle das condições experimentais e análise da variabilidade [2, 8, 3].

6 Conclusão

A versão otimizada apresentou menor tempo de execução em todos os tamanhos avaliados sem recorrer a paralelismo entre núcleos. O mesmo cálculo e o mesmo layout de dados foram mantidos, enquanto a otimização concentrou-se no kernel por meio de AVX, desenrolamento do laço, acumuladores independentes e `restrict`.

Na execução experimental com $N = 16384$, o fator de aceleração variou aproximadamente entre 1,22 e 1,73. O maior ganho ocorreu em $T = 1024$, com redução de aproximadamente 42,3% no tempo mediano. Em $T = 2048$, o fator diminuiu para cerca de 1,45, acompanhado por menor vazão e maior custo por elemento na versão otimizada. Em $T = 4096$, o fator ficou em aproximadamente 1,50, correspondendo a redução de 33,5% no tempo mediano. A versão otimizada apresentou menor custo por elemento em todos os tamanhos, com mínimo de aproximadamente 0,661 ns/elemento em $T = 1024$.

Os resultados mostraram ganhos de desempenho com a otimização, mas esses ganhos variam conforme o tamanho da entrada e não seguem uma tendência monotônica. As medições coletadas permitem quantificar os ganhos e sua variabilidade, mas não permitem atribuir as oscilações observadas a um mecanismo microarquitetural específico, necessitando de uma experimentação adicional mais aprofundada.

Referências

- [1] G. Buttazzo and G. Lipari, “Ptask: An educational C library for programming real-time systems on Linux,” in *2013 IEEE 18th Conference on Emerging Technologies & Factory Automation (ETFA)*, pp. 1–8, 2013.
- [2] T. Hoefer and R. Belli, “Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–12, 2015.
- [3] C. Curtsinger and E. D. Berger, “Stabilizer: Statistically sound performance evaluation,” *ACM SIGARCH Computer Architecture News*, vol. 41, no. 1, pp. 219–228, 2013.
- [4] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, Version 050, 2026. Disponível em: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel64-and-ia32-architectures-optimization.html>.
- [5] M. Kerrisk and Linux man-pages contributors, “perf_event_open(2) – set up performance monitoring,” *Linux man-pages*, 2026. Disponível em: https://man7.org/linux/man-pages/man2/perf_event_open.2.html.
- [6] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!,” *ACM SIGPLAN Notices*, vol. 44, no. 3, pp. 265–276, 2009.
- [7] H. Schildt, *C++: The Complete Reference*. McGraw-Hill, 1998.
- [8] T. Kalibera and R. Jones, “Rigorous benchmarking in reasonable time,” in *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, pp. 63–74, 2013.